WABA Group — Lakehouse financier multi-pays : write-up technique

Challenge Data Engineer Artefact · Mohamed Kindo · dépôt : `github.com/mkindo19/waba-lakehouse` Niveaux 1 à 4 réalisés. Le journal détaillé des 34 décisions est dans `docs/DECISIONS.md` (références « Dxx »).

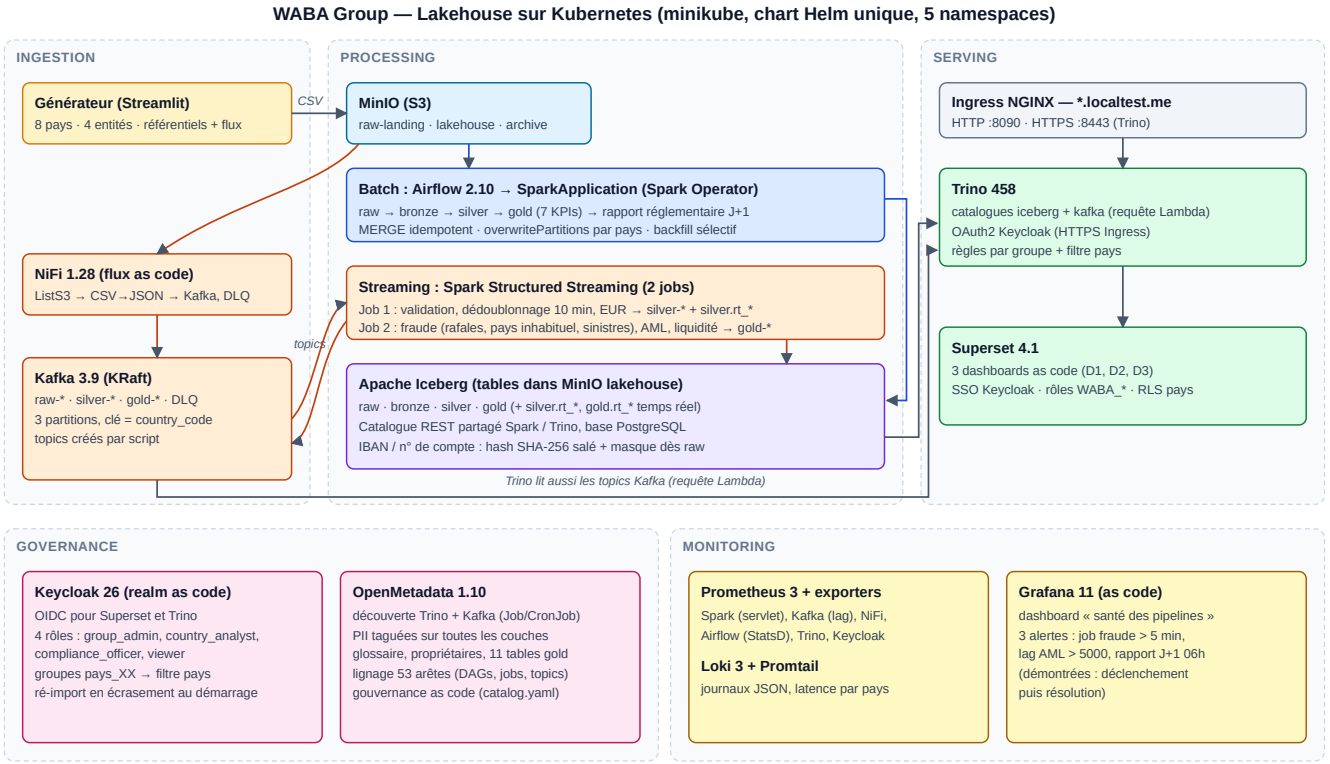
1. Contexte et objectifs

WABA Group réunit quatre métiers (banque, assurance, mobile money, microfinance) dans huit pays d'Afrique de l'Ouest : sept en zone UEMOA (XOF, BCEAO / CIMA) et le Ghana (GHS, Bank of Ghana / NIC). La plateforme doit consolider ces données hétérogènes, produire les indicateurs de pilotage et réglementaires (NPL, loss ratio, ARPC, délais de sinistres, rapport quotidien J+1), détecter fraude et blanchiment en temps réel, et protéger les données personnelles (IBAN, numéros de compte, montants individuels).

Trois principes ont guidé tous les choix :

- **Tout est code et rejouable** : flux NiFi, dashboards Superset et Grafana, realm Keycloak, gouvernance OpenMetadata, règles d'accès Trino. Une réinstallation reproduit la plateforme à l'identique, sans clic.
- **Idempotence** : chaque job peut être relancé sans doublon, ce qui permet les reprises et les backfills par pays.
- **Sécurité par défaut** : aucun secret dans le dépôt, pseudonymisation dès la première écriture, moindre privilège pour chaque composant.

2. Architecture



Stockage. Les fichiers déposés dans MinIO (`raw-landing`) sont écrits en tables **Apache Iceberg** dans le bucket `lakehouse`. Le choix de la couche table est central : Iceberg apporte les transactions ACID, le `MERGE` (idempotence), l'écrasement de partitions (recalcul par pays) et le time travel, en restant lisible par Spark et Trino via un **catalogue REST partagé**. Organisation médaillon :

Couche	Rôle	Partitionnement
`raw`	miroir validé des fichiers, rejets tracés avec leur motif	pays + date métier
`bronze`	journal des chargements, rejouable et auditable	pays + date d'ingestion
`silver`	dédoublonné, joint aux référentiels, converti en EUR, anomalies signalées	pays + mois
`gold`	7 KPIs, rapport réglementaire J+1, alertes temps réel (`rt_*`)	pays

Batch. Quatre DAGs Airflow chaînés (ingestion toutes les 10 min si nouveaux fichiers → silver → gold, puis rapport réglementaire à 00h30 UTC). Chaque tâche lance une `SparkApplication` (Spark Operator) : pas de cluster Spark permanent, la mémoire n'est occupée que pendant les jobs. Un paramètre `country_code` propage un **backfill sélectif** d'un DAG à l'autre ; seules les partitions du pays sont réécrites (vérifié par requête de contrôle).

Streaming. NiFi détecte les fichiers (ListS3, 10 s) et publie une ligne = un message JSON dans Kafka (clé = pays). Le **job 1** Spark Structured Streaming applique les mêmes règles de qualité que le batch (rejets vers une DLQ), dédoublonne sur 10 minutes et écrit à la fois `silver-*` (Kafka) et `silver.rt_*` (Iceberg). Le **job 2** détecte les rafales de retraits (fenêtres glissantes 5 min / pas 1 min), les envois depuis un pays inhabituel, les sinistres anormaux, les opérations AML et les tensions de liquidité, et publie dans `gold-*` + `gold.rt_*`.

Architecture Lambda. La vue batch (exacte) et la vue temps réel (fraîche) sont séparées. Trino lit les deux (catalogues `iceberg` et `kafka`). La requête de l'énoncé tourne telle quelle ; une version corrigée évite le double comptage en ne prenant le temps réel que pour les jours postérieurs au dernier calcul batch du pays (D24).

Exposition. Trino est le moteur SQL unique ; Superset présente trois dashboards (performance commerciale, risque et conformité, mobile money) avec un filtre pays commun. L'ensemble tourne sur **minikube**, installé par **un seul chart Helm** (`make k8s-install`) qui répartit les composants dans cinq namespaces (ingestion, processing, serving, governance, monitoring).

3. Choix techniques et compromis

Choix	Pourquoi	Compromis / alternative écartée
Pseudonymisation **dès raw** : `iban` / `account_number` → hash SHA-256 salé + valeur masquée (D3)	Aucune donnée en clair dans le lakehouse ; le hash garde jointures et comptages	Les valeurs en clair ne vivent que dans la zone d'atterrissage, avant archivage
`MERGE` Iceberg pour les événements, `overwritePartitions` pour silver / gold (D4, D12)	Rejouer un job ne crée aucun doublon ; backfill d'un pays sans toucher aux autres	Silver / gold recalculés par pays plutôt qu'incrémentaux : simple et sûr, plus coûteux à grande échelle
Catalogue Iceberg REST sur **PostgreSQL** (D31)	Écritures concurrentes batch + streaming sans conflit	SQLite gardé en Docker Compose (niveaux 1-3). En production : Polaris ou Lakekeeper avec OAuth2
Chart Helm « maison » plutôt que les charts officiels (D26)	Un namespace par domaine (les charts publics restent dans le namespace de la release), probes et ressources maîtrisées	Plus de YAML à maintenir ; Spark Operator reste un sous-chart officiel
Opérateur Airflow maison pour `SparkApplication` (D28)	Le `SparkKubernetesOperator` du provider ne retrouve pas le driver sans étiquettes qu'il n'ajoute pas : une tâche pouvait passer « réussie » sur un job Spark en échec (lu dans son code)	~100 lignes à maintenir
Retard des jobs Spark mesuré par un groupe Kafka dédié (D30)	Spark ne committe pas ses offsets : sans cela, le retard est invisible. Si le job tombe, les offsets se figent et le retard grandit, ce qui déclenche l'alerte	Le groupe n'existe qu'après le premier micro-lot traité
OpenMetadata sans son chart officiel (D32)	Le chart embarque un second Airflow (~2 Go) ; la découverte tient dans un Job / CronJob	Le lignage table → table est déclaré à côté du code (53 arêtes), pas capté à l'exécution
NiFi 1.28 plutôt que 2.x (D19)	HTTP possible en local, processeurs Kafka / S3 inclus ; flux déployé par script via l'API	Migration 2.x (HTTPS natif) à prévoir
Une ligne CSV = un message Kafka, colonnes en texte (D19)	NiFi ne devine aucun type ; validation et typage uniquement dans Spark, comme en batch	Messages plus nombreux qu'un envoi par fichier

Les **KPIs** reposent sur des hypothèses documentées (D15) : NPL pondéré par l'encours (défaut ou plus de 90 jours de retard), ARPC = commissions + intérêts / clients actifs du mois, SLA sinistres de 30 jours ouvrés (IARD) et 45 (Vie). Conversion EUR : parité fixe XOF, taux GHS paramétrable et tracé dans `silver.fx_rates`. Conformément à l'énoncé, la Guinée est traitée en XOF (D1, une ligne à changer pour passer en GNF).

4. Gouvernance et sécurité

Identité : Keycloak. Un realm `waba` généré par script (`keycloak/gen_realm.py`) et ré-importé en écrasement à chaque démarrage : le dépôt reste la source de vérité. Les quatre rôles de l'énoncé sont portés par des groupes fonctionnels ; le pays d'un analyste est un groupe `pays_XX`. Les jetons transmettent `roles` et `groups`.

Rôle Keycloak	Superset	Trino (accès direct)
`group_admin`	Admin : tout, SQL Lab	tous catalogues, lecture / écriture
`country_analyst`	3 dashboards, lignes de **son pays** (RLS)	silver + gold, filtre `contains(current_groups(), 'pays_' || country_code)`
`compliance_officer`	D2 Risque & Conformité, tous pays	9 tables gold réglementaires
`viewer`	D1 et D3 en lecture seule : ni exploration, ni « drill to detail », ni export	7 tables gold agrégées ; ni silver, ni événements individuels, ni Kafka

Les droits sont appliqués aux deux portes d'entrée : dans Superset (rôles `WABA_*` et filtres RLS créés par script, D33) et dans Trino (règles par fichier, D34). Un analyste sans groupe pays ne voit **aucune** ligne. Vérifié sur le cluster : `analyste.ci` ne voit que la Côte d'Ivoire (26 614 transactions silver, NPL CI seul), `lecteur` est refusé sur silver, sur `rt_aml_events` et sur Kafka.

Données personnelles. Hash + masque dès raw ; OpenMetadata marque automatiquement les colonnes sensibles sur toutes les couches (`PII.Sensitive`, classification `ProtectionDonnees`) ; le glossaire financier, les propriétaires (5 équipes) et les étiquettes réglementaires BCEAO / CIMA documentent les 11 tables gold ; le lignage relie fichiers, DAGs, jobs streaming, topics et tables.

Secrets. Uniquement dans `.env` (généré avec des valeurs aléatoires, jamais versionné), transformé en un Secret Kubernetes par namespace, limité aux clés dont ce namespace a besoin. Comptes de service à droits minimaux (Airflow limité aux `SparkApplication` de son namespace, Grafana en lecture seule sur une seule table d'Airflow).

5. Observabilité

Prometheus collecte une source par composant (servlet Spark, kafka-exporter, NiFi, Airflow via StatsD, Trino, Keycloak) ; Loki centralise les journaux JSON des jobs (dont la latence par pays). Grafana, entièrement provisionné par fichiers, affiche le dashboard « santé des pipelines » (18 panneaux) et porte les **trois alertes** de l'énoncé. Elles ont été **réellement déclenchées puis résolues** en démonstration : job fraude cassé pendant 50 min, retard AML monté à 45 700 messages, rapport réglementaire absent après 06h00 UTC (requête SQL sur la base Airflow : un compteur Prometheus ne verrait pas un DAG qui n'a jamais tourné).

6. Problèmes rencontrés et résolus

- **Écritures concurrentes sur le catalogue** : batch et streaming committant ensemble, le catalogue SQLite répondait `database is locked` ; les topics avaient reçu 17 fois les mêmes messages. Correctifs successifs : verrou de commit et registre des micro-lots publiés (D22), puis migration du catalogue vers PostgreSQL sans réécrire une seule table (D31). C'est la supervision qui a révélé le problème (taux d'erreur des DAGs, retard qui ne montait plus).
- **Conflits de commit Iceberg** entre requêtes d'un même job (snapshot en cache) : `REFRESH TABLE` avant chaque `MERGE` et reprise automatique (D25).
- **CPU saturé à 90 % de réservation** pour 13 % d'usage réel : executors en attente. Réservation (`coreRequest`) séparée du parallélisme (`cores`) : 59 % (D30).
- **Executor tué par manque de mémoire** (`OOMKilled`) sur Kubernetes alors qu'il passait en Docker Compose : la limite du pod doit couvrir les workers Python et la mémoire hors tas (D28).
- **SSO derrière l'Ingress** : le contrôleur annonce le port 80 alors que le navigateur est sur 8090, ce qui cassait l'URL de retour OIDC de Superset (corrigé en ignorant ce port). Trino n'accepte OAuth2 que sur une connexion sécurisée et pose des cookies `__Secure-*` : le TLS est terminé par l'Ingress (`:8443`), Trino voyant la requête comme HTTPS. Ces deux points ont été établis en lisant le code de Trino 458, qui ne pouvait pas être lancé dans l'environnement de préparation (D33, D34).

Méthode générale : chaque composant a d'abord été validé hors cluster (Superset et Keycloak pilotés par Playwright, OpenMetadata et le catalogue sur PostgreSQL local, configurations Prometheus / Loki / Promtail vérifiées par leurs outils), puis sur minikube.

7. Limites connues et passage en production

- **Infrastructure locale** : un nœud minikube (8 cœurs, 24 Go, ~75 % de mémoire utilisée), Kafka à un broker (production : 3 brokers, réplication 3), pas de haute disponibilité. Production : cluster managé, autoscaling des executors Spark, stockage objet managé à la place de MinIO (l'image communautaire n'est plus maintenue, D9).
- **Transport** : HTTP sur le réseau interne et certificat auto-signé à l'Ingress. Production : HTTPS de bout en bout, certificats gérés (cert-manager), `sslRequired: external` dans Keycloak.
- **Comptes de service Trino** (Superset, OpenMetadata, Prometheus) authentifiés par leur seul nom en HTTP interne, limités à une liste fermée de noms. Production : mot de passe ou JWT, NetworkPolicy, port HTTP fermé. Superset interroge Trino avec son compte de service : les droits fins sont appliqués côté Superset (impersonation à étudier).
- **Secrets** : `.env` sur le poste. Production : External Secrets Operator + Vault.
- **Livraison « au moins une fois » côté Kafka** : un rejeu peut republier une alerte (même clé). Production : producteur transactionnel ou topic compacté sur `alert_id`.
- **Lignage déclaratif** : à remplacer par OpenLineage dans Spark (lignage au niveau colonne). Notifications d'alertes non branchées (pas de SMTP / Teams en local).
- **Données simulées** : seuils de fraude, de liquidité et SLA calibrés pour la démonstration, à valider avec les métiers. CI/CD (tests, lint, `helm lint`, construction des images) à automatiser dans un pipeline.